

Dictionary

A Dictionary represents a group of elements in the form of key value pairs.

Dictionary in Python is an unordered collection.

Dictionaries are mutable so we can modify it's item, without changing their identity.

Dictionaries are represented using curly bracket { }.

Ex:-

```
stu = {101: 'Rahul', 102: 'Raj', 103: 'Sonam' }
```

```
fees= {'rahul':2000, 'raj':3000, 'sonam':8000}
```

Creating an Empty Dictionary

Syntax:- `dict_name = { }`

Ex:- `fees = { }`

Creating a Dictionary

A dictionary is created in the form of key-value pair where keys can't be repeated and must be immutable and values can be of any datatype and can be duplicated.

keys are case sensitive.

Syntax:- `dict_name = {key1:value1, key2:value2,...}`

Ex:-

```
stu = {101: 'Rahul', 102: 'Raj', 103: 'Sonam' }
```

```
fees = {'rahul':2000, 'raj':3000, 'sonam':8000 }
```

```
fees = {  
    'rahul': 2000,  
    'raj': 3000,  
    'sonam': 8000  
}
```

Key Rules

While writing key we must follow the following rules:

- Key should be unique.
- If we mention same key again, the old key will be overwritten.
- Key should be immutable type ex:- integer, string or tuple.
- We can not use list or dictionary as key.

Accessing Dictionary

We can access the value of a dictionary by referring to its key name, inside square brackets.

```
stu = {101: 'Rahul', 102: 'Raj', 103: 'Sonam' }
```

```
fees = {'rahul':2000, 'raj':3000, 'sonam':8000}
```

```
print(stu[101])
```

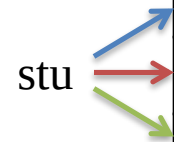
```
print(stu[102])
```

```
print(stu[103])
```

```
print(fees['rahul'])
```

```
print(fees['raj'])
```

```
print(fees['sonam'])
```



[101]	Rahul
[102]	Raj
[103]	Sonam



['rahul']	2000
['raj']	3000
['sonam']	8000

Modifying

We can modify the existing value of key by assigning a new value.

```
stu = {101: 'Rahul', 102: 'Raj', 103: 'Sonam' }
```

```
stu[102] = 'Python'
```

Inserting/Adding new item

We can add an item to dictionary just by mentioning a new key-value pair into an existing dictionary.

If we mention a key which already exists in the dictionary then the value gets updated/modified rather than adding a new item.

The new item may be added at any place in the dictionary as dictionary is an unordered collection.

```
stu = {101: 'Rahul', 102: 'Raj', 103: 'Sonam' }
```

```
stu[104] = 'Geekyshows'
```

Deletion

We can delete an item of dictionary or entire dictionary using del statement.

```
stu = {101: 'Rahul', 102: 'Raj', 103: 'Sonam' }
```

Deleting an item

```
del stu[102]
```

Deleting entire Dictionary

```
del stu
```


Testing Key

We can check whether a key already exists in the dictionary or not, for this purpose we use membership operator.

```
stu = {101: 'Rahul', 102: 'Raj', 103: 'Sonam' }
```

```
101 in stu          # If exists returns True
```

```
104 not in stu      # If not exists returns True
```

Clear () Method

This method is used to remove all the elements from the dictionary.

Syntax:- dict.clear()

Ex:- stu.clear()

copy () Method

This method is used to copy all the elements from the existing dictionary into a new dictionary.

Syntax:- dict.copy()

Ex:-

```
new_stu = stu.copy( )
```

fromkeys () Method

This method is used to create a new dictionary with the specified keys and values.

Syntax:- dict.fromkeys(keys, value)

Ex:-

```
key = (101, 102, 103)
```

```
value = 'GeekyShows'
```

```
new_stu = dict.fromkeys(key, value)
```

get () Method

This method returns the value of the specified key.

If key is not found then it will return none or default value.

Syntax:- dict_name.get(key, defaultvalue)

Ex:-

```
stu.get(104)
```

```
stu.get(104, 'GeekyShows')
```

items () Method

This method returns an object that contains key-value pairs of dictionary.

The pairs are stored as tuples in the object.

Syntax:- dict_name.items()

Ex:-

```
stu.items()
```

keys () Method

This method returns a sequence of keys from the dictionary .

Syntax:- dict_name.keys()

Ex:- stu.keys()

values () Method

This method returns a sequence of values from the dictionary .

Syntax:- dict_name.values()

Ex:- stu.values()

update () Method

This method is used to update the dictionary with the specified key value pair.

Syntax:- dict_name.update(iterable)

Ex:- stu.update({105: 'Gupchup'})

pop () Method

This method is used to remove the item with specified key.

It returns the removed item's value.

If key is not found then the a default value is returned.

If key is not found and default value is not given then shows KeyError.

Syntax:- dict_name.pop(key, defaultvalue)

Ex:- stu.pop(101)

Ex:- stu.pop(110, 'GeekyShows')

popitem () Method

This method is used to remove the item which was last inserted into the dictionary.

It returns the removed item in the form of tuple, Pairs are returned in LIFO order.

Syntax:- dict_name.popitem()

Ex:- stu.popitem()

setdefault () Method

This method returns the value of the specified key.

If key is not found then it inserts key with the specified value.

Syntax:- dict_name.setdefault(key, value)

Ex:- stu.setdefault(109, 'Rohit')